

CSCE 713 SOFTWARE SECURITY · MODULE 01 · LECTURE NOTES

Why Software Has Vulnerabilities

Ordinary bugs with an adversary attached.

MODULE OUTCOMES

- Explain what distinguishes a vulnerability from a bug.
- Explain the input and trust boundary framing.
- Explain what makes a defect severe.
- State the course's authorisation constraint.
- Assess a defect's exploitability honestly.

1 The distinction

A VULNERABILITY IS A BUG THAT AN ADVERSARY CAN REACH AND BENEFIT FROM

The code defect is exactly the same thing. What makes it a vulnerability is that an untrusted party can influence the input that triggers it, and gains something when it fires. Nothing about the code changes.

So three things are required: a defect, a path from attacker-controlled input to that defect, and a consequence the attacker actually wants. Remove any one of the three and you have an ordinary bug — which still deserves fixing, and on a different schedule.

Which means the identical defect differs enormously in severity by context. A buffer overflow in a parser reading network data is critical; byte-for-byte identical code reading a compile-time constant is a latent defect waiting for somebody to reuse the function. This is why severity cannot be read off the code alone (§3).

And it means the techniques that find correctness defects find vulnerabilities — the difference lies in what you treat as an input and who you imagine supplying it, which is the whole of §2 and is the single most transferable idea in the course. A fuzzer is a test generator with a hostile imagination.

- The language permits it. C and C++ make memory errors expressible, and a language in which they are not expressible does not have them (Module 02 §4) — which is an observation about expressiveness rather than about programmers.
- The interface invites it. An API that can be called incorrectly will eventually be called incorrectly — which is CSCE 711 Module 01 §4's misuse-resistance argument in its general form, and applies to every API rather than only to cryptographic ones.

- **The assumption was undocumented.** A function that requires its input to have been validated, and does not say so, will eventually be called with unvalidated input by somebody who had no way to know — frequently the original author, two years later.
- **The composition was never considered.** Two components, each entirely correct in isolation, with subtly incompatible assumptions at the seam between them — which is where a great many severe defects live and where no single-component analysis finds them.
- **And the code outlived its context** — written for a trusted local caller, later exposed to a network, with the original trust assumption still embedded and no longer true. **Note what is not on this list: programmer carelessness.** These defect classes recur across decades, across languages, and across extremely capable teams, which is precisely what makes them a property of the tools rather than of the people.

2 Inputs and trust boundaries

YOUR INPUT SURFACE IS LARGER THAN THE PARAMETERS OF YOUR FUNCTIONS

The obvious inputs: network requests, files, command-line arguments, environment variables, standard input. **Everyone enumerates these.**

The less obvious: filenames and file metadata, database contents, another service's response, a configuration file, a message from a queue, the system clock, the contents of a directory listing, an environment variable set by a parent process, and the terminal's own escape sequences.

And the one most often missed entirely: data your own system wrote earlier — which is attacker-controlled if the attacker could influence what was stored, and is then read back and trusted because 'it came from our database'. **This is the mechanism behind stored injection of every kind.**

So the question to ask of every piece of data is 'who could have influenced this, and what did we assume about it' — which is CSCE 701 Module 01's trust boundary analysis, applied at function granularity rather than at system granularity, and is the same technique at a different scale.

1. ENUMERATE THE ENTRY POINTS
everywhere untrusted data enters the program
2. TRACE EACH ONE FORWARD
where does it go? what parses it? what indexes with it? what allocates based on it? what is it interpreted by, downstream?
3. AT EVERY USE, ASK WHAT WAS ASSUMED
a length that fits, a non-null pointer, a valid UTF-8 string, a number in range, a path without ".", a trusted serialised object, a terminating null byte
4. THEN ASK WHETHER THE ASSUMPTION IS CHECKED
and specifically, checked on THIS path -- a check at one call site does not protect another, and the defect is usually at the call site nobody listed

THIS IS TAINT ANALYSIS DONE BY HAND, and Module 07 automates it imperfectly. Doing it manually first is what makes the tool's output readable.

Step 4's 'on this path' qualifier is where the real defects hide. The validation exists, it is correct, and one caller bypasses it — a refactor introduced a second entry point, an optimisation skipped the wrapper, a new feature called the internal function directly. **Which is why the question is never 'is this input validated' but 'is it validated on every path that reaches this use'**, and that is a question about the call graph.

3 Severity

Factor	The question	Why it matters
Reachability	Can untrusted input actually get there?	Unreachable means it is not a vulnerability (§1) — and this is where most triage effort belongs.
Pre-authentication	Is it reachable before any login?	The single largest severity multiplier — see the note.
Primitive gained	A crash, an out-of-bounds read, an out-of-bounds write, or control of execution?	A write is far worse than a read, and control of a function pointer worse again.
Privilege context	What does the affected process actually hold?	Least privilege bounds the damage (CSCE 701 Module 03) — which is why it is worth the trouble.
Reliability	Deterministic, or dependent on a race or a heap layout?	Affects practical exploitability and therefore the realistic urgency.

Pre-authentication reachability is the factor that dominates, and it is systematically under-weighted. A defect behind authentication requires the attacker to obtain an account first, which changes the adversary set from 'anyone on the internet' to 'anyone who can register or steal credentials' — a difference of several orders of magnitude in exposure. **Students reliably over-weight the primitive gained and under-weight how the input arrives.**

AND BE HONEST ABOUT EXPLOITABILITY IN BOTH DIRECTIONS

Overclaiming wastes effort. A crash is not automatically code execution — modern mitigations (Module 12) make a great many memory defects genuinely hard to exploit, and **treating every crash as critical exhausts the remediation budget on the wrong things** (CSCE 701 Module 12's base rate problem, in a different setting).

And underclaiming is worse. 'Just a crash' has repeatedly turned out to be fully exploitable once somebody capable spent a week on it — so **'not exploitable' is a strong claim that requires real analysis**, and it is usually asserted without any.

So the honest position is to state precisely what you established: 'reachable from unauthenticated input, causes an out-of-bounds write of attacker-controlled length at an attacker-influenced offset, exploitability not analysed'. Every clause is checkable.

Which is the program's closing rule arriving in Module 01 — and a defect report in that form is immediately actionable, while both 'critical RCE' and 'low, just a crash' require the reader to redo your analysis from scratch. Module 13 develops this into the course's closing position.

4 The constraint, and the course

YOUR OWN CODE, AND DELIBERATELY VULNERABLE TARGETS. NOTHING ELSE.

Every exercise in this course is performed on code you own or on a training target built for the purpose — OWASP Juice Shop, WebGoat, the fuzzer test suite, your own projects, your own build of open-source software.

Testing systems you are not authorised to test is unlawful in most jurisdictions regardless of your intent — and the absence of harm is not a defence, nor is curiosity, nor is having found something real. This is settled and not a grey area.

And when you find a defect in software you use, disclose it responsibly: report privately to the maintainer, allow reasonable time for a fix, and coordinate publication (Module 13 §2, which treats this as a professional obligation rather than a courtesy).

This constraint is identical to the one CSCE 701 states, and it applies directly to Project 2, where you will assess a real project — fuzzing your own local build of open-source software is entirely fine, and probing somebody's running deployment of it is not, and the distinction is the whole of the rule.

- **Modules 02 through 06: the defect classes — memory safety, integers and type confusion, injection, concurrency, and deserialisation**, which together account for the great majority of severe defects in the published data.
- **Modules 07 through 09: the techniques** — static analysis, fuzzing, and dynamic sanitisers, **with their theoretical limits stated honestly** (CSCE 627's undecidability results, which bound what any tool can promise).
- **Modules 10 through 12: the practice** — the development lifecycle, systematic code review, and the exploit mitigations you rely on when the defect is already shipped and cannot be removed.
- **And Module 13: what an assessment actually establishes**, which is the course's closing position and the form of Project 2's final claim.
- **The ordering is deliberate: you cannot read a tool's output without already knowing the classes it is looking for. Defect classes before tooling** — because a static analyser's findings are genuinely unreadable to somebody who does not already know what a use-after-free is, and the commonest failure of static analysis adoption is handing the output to people who cannot triage it (Module 07 §3).

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Dowd, McDonald & Schuh, chapters 1 and 2	§1 and §2's method — the best treatment of reading code adversarially that exists. Library copy.

Resource	What it covers for this module
The CWE Top 25 and its methodology (free)	§3's severity factors and the evidence for Modules 02 through 06's ordering — including how the ranking is computed, which is worth understanding.
CVSS specification (free)	§3 as the industry's scoring system — useful, widely required, and read CSCE 701 Module 12 on why a single number misleads.
MIT 6.858, the first lectures (free, OCW)	§1 and §2 as lectures , free, with the threat model framing developed carefully.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Take a bug you have fixed** and determine whether it was a vulnerability, using the three-part test.
- 2 **Find a defect whose severity depends entirely on context**, and describe both contexts.
- 3 **Enumerate every input** to a program you wrote — including the non-obvious ones.
- 4 **Find one place you trust data your own system wrote.**
- 5 **Run Part 2's four steps** on one entry point, by hand, and record what you assumed at each use.
- 6 **Find one assumption that is checked on one path and not another.**
- 7 **Rank five defects by Part 3's factors** and compare against your intuition.
- 8 **Rewrite a severity claim** in the honest form.
- 9 **Install a deliberately vulnerable target** and confirm it runs locally.
- 10 **Write the authorisation constraint in your own words**, and keep it for Project 2.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What three things does a vulnerability require?
- 2 Why does the same defect differ in severity by context?
- 3 Give five reasons defects exist, and say what is not on the list.
- 4 Name five non-obvious inputs.
- 5 Why is data your own system wrote an input?
- 6 Give Part 2's four steps and say where defects hide.
- 7 Name five severity factors and which dominates.

- 8 Why is overclaiming bad, and why is underclaiming worse?
- 9 Give the honest form of a defect report.
- 10 State the authorisation constraint and how it applies to Project 2.